

Based on OpenCV and Support Vector Machine

Traffic Sign Classification and Recognition System

Design and Implementation

Computer Vision Course Design Report

Core Tech	OpenCV / HOG / HSV / SVM / KNN / RF / scikit-learn / PyQt5
Task	Traffic sign classification (core) + traditional color/contour detection (extension)
Dataset	GTSRB - 43 classes, 39,209 training samples, 12,630 test samples
Environment	Python 3.12 / Windows 11 / CPU-only training and inference
Purpose	Course design proposal, system design, experiment implementation, defense presentation

Date: July 11, 2026

Table of Contents

- ☐ 1. Project Overview and Scope
- ☐ 2. System Architecture – Data Layer -> Algorithm Layer -> Application Layer
- ☐ 3. Technology Stack and Runtime Environment
- ☐ 4. Dataset Selection and Sample Distribution
- ☐ 5. OpenCV Image Preprocessing Module
- ☐ 6. Feature Extraction: HOG, HSV Histogram and Feature Fusion
- ☐ 7. Classification Models: SVM / KNN / Random Forest
- ☐ 8. Training Pipeline and Hyperparameter Selection
- ☐ 9. Model Evaluation and Results Analysis
- ☐ 10. SVM vs KNN vs RF Multi-Model Comparison
- ☐ 11. Error Samples and Confusable Class Analysis
- ☐ 12. PyQt5 Graphical User Interface Design
- ☐ 13. Traditional Vision Detection Extension (SignDetector)
- ☐ 14. Improvement Directions and Outlook
- ☐ 15. Limitations and Conclusion

This course design is based on the GTSRB dataset with OpenCV HOG + SVM as the core pipeline. It covers data reading, preprocessing, feature engineering, model training, multi-model comparison, error analysis, GUI interaction, and traditional detection extension.

1. Project Overview and Scope

With the rapid development of Intelligent Transportation Systems (ITS) and driver assistance technologies, automatic traffic sign recognition has become a fundamental research topic in computer vision. This project follows a traditional machine learning approach using OpenCV for image preprocessing and feature extraction, with SVM as the core classifier, building a complete traffic sign classification and recognition system.

The system core focuses on traffic sign classification: input is a single cropped traffic sign image, output is class ID, class name and confidence. Additionally, a traditional detection extension locates candidate sign regions in full road scenes using HSV color masking, morphological operations, and contour geometric filtering before classification.

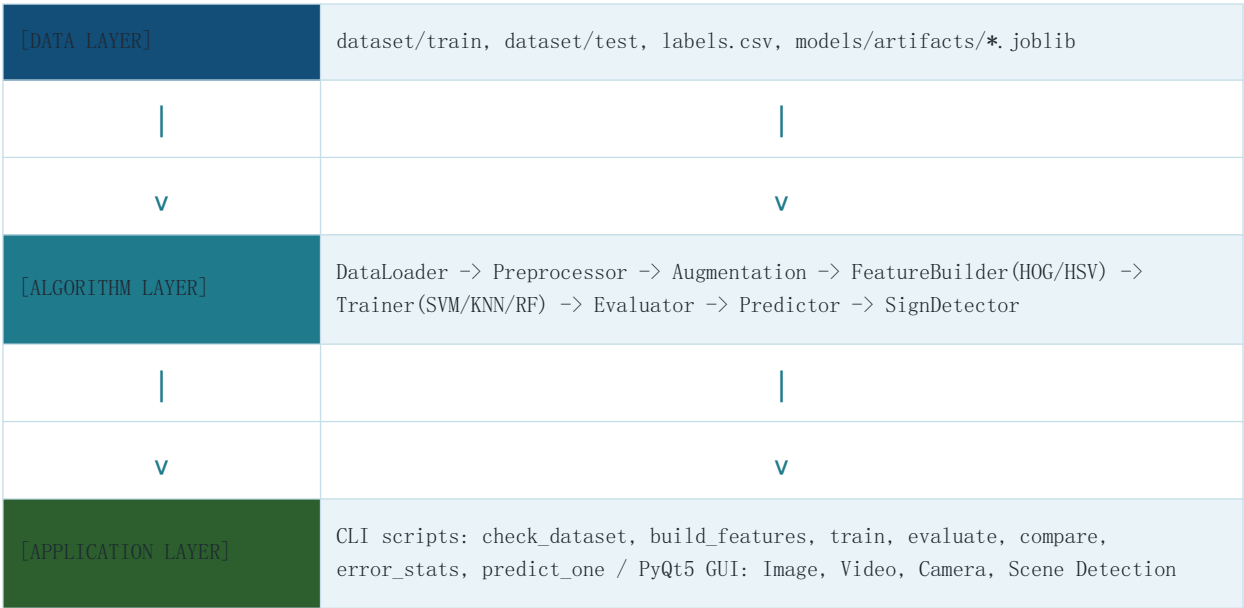
Item	Classification Task	Detection Extension
Input	Cropped single sign (64x64)	Road scene image, video frame, or camera feed
Output	Class ID, name, confidence	BBox (x,y,w,h), class, confidence
Tech Focus	HOG/HSV features, SVM multi-class	HSV mask, morphology, contour filtering + classification
Priority	Required (core)	Optional / bonus

The project forms a complete closed loop: dataset management -> image preprocessing -> feature extraction -> model training -> model evaluation -> real-time recognition.

2. System Architecture

The system adopts a three-layer architecture: Data Layer handles dataset management and model artifact persistence; Algorithm Layer implements preprocessing, feature building, training, evaluation, and recognition; Application Layer provides CLI scripts and PyQt5 GUI. Standardized data interfaces (NumPy arrays, joblib bundles, Pandas DataFrames) connect the layers.

2.1 Overall Architecture Diagram



2.2 Module Dependency Flow

Core data path: data loading -> preprocessing -> feature building -> model training -> model evaluation -> model persistence -> prediction inference.

```
DataLoader.load_train_data()      # (images, labels, class_ids)
|
Preprocessor(img_bgr)             # -> grayscale uint8 (64x64)
| [opt] Augmentation.apply_random(img, p=0.5)
FeatureBuilder.extract_batch()    # -> (N, D) float32
| StandardScaler.fit_transform()
SVC / KNeighbors / RandomForest.fit()
|
Evaluator -> confusion_matrix.png, metrics.json, errors.csv
| joblib.dump()
Predictor(predict_proba) -> {class_id, class_name, confidence}
```

3. Technology Stack and Runtime Environment

3.1 Language and Development Environment

Python 3.12.7 (Anaconda) / Windows 11. OpenCV provides efficient image I/O and feature extraction, scikit-learn offers comprehensive traditional ML APIs, PyQt5 enables rapid desktop GUI development. All modules run on CPU without GPU dependency.

3.2 Core Dependencies

Library	Version	Primary Role
opencv-python	4. x	Image I/O, preprocessing, HOGDescriptor, video/camera
NumPy	1. x	Array operations, feature concatenation, image matrices
scikit-learn	1. x	SVC/KNN/RF classifiers, StandardScaler, stratified split, metrics
Matplotlib	3. x	Confusion matrix, class distribution charts, comparison plots
Pandas	2. x	Label CSV parsing, DataFrame statistics export
Joblib	1. x	Model bundle serialization/deserialization
PyQt5	5.15	QMainWindow, QThread, signal/slot, QTimer

3.3 Environment Setup

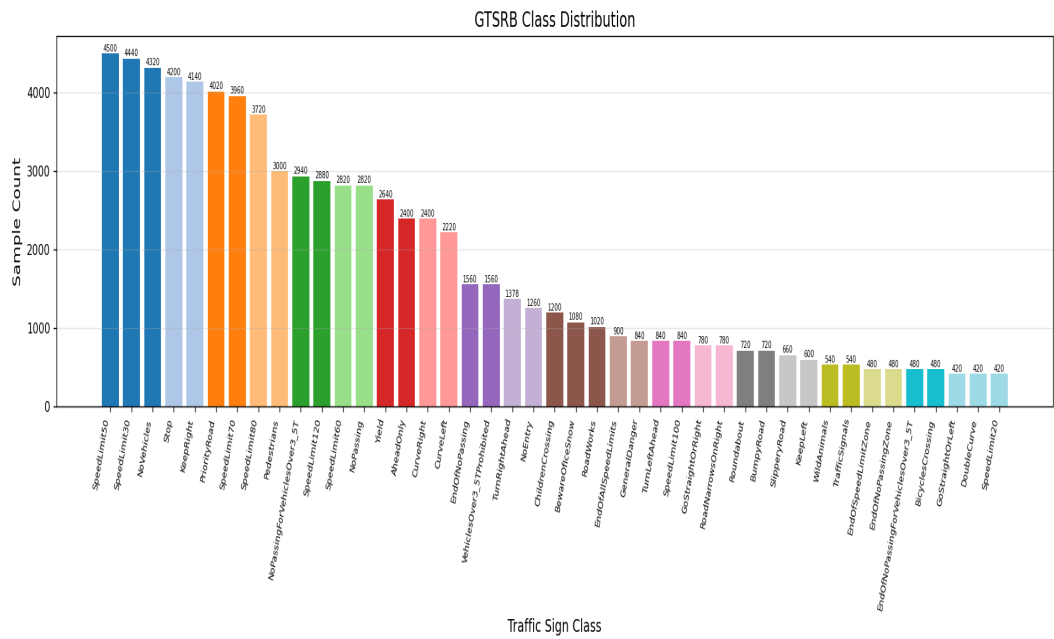
```
conda create -n traffic-sign python=3.12
conda activate traffic-sign
pip install opencv-python numpy pandas matplotlib scikit-learn joblib PyQt5
```

4. Dataset Selection and Sample Distribution

GTSRB (German Traffic Sign Recognition Benchmark) is the classic benchmark for traffic sign classification. It contains 43 classes, 39,209 training images (varying 15x15 to 250x250 sizes), and 12,630 test images (with ROI annotation CSV). Classes include speed limits, prohibitory signs, warning signs, and other signs.

4.1 Class Sample Distribution

Below is the sample count distribution of the 43 GTSRB training classes. Note the class imbalance: some classes have 2,000+ samples while minority classes have only a few hundred. This may affect minority class recall.



(Generated by scripts/check_dataset.py at models/artifacts/class_distribution.png)

4.2 GTSRB 43 Class Labels

ID	Name	ID	Name	ID	Name
0	Speed limit 20	1	Speed limit 30	2	Speed limit 50
3	Speed limit 60	4	Speed limit 70	5	Speed limit 80
6	End 80 limit	7	Speed limit 100	8	Speed limit 120
9	No overtaking	10	No truck overtaking	11	Intersection priority
12	Priority road	13	Yield	14	Stop
15	No vehicles	16	No trucks	17	No entry
18	General caution	19	Left curve	20	Right curve
21	Double curve	22	Bumpy road	23	Slippery road
24	Road narrows	25	Road work	26	Traffic signals
27	Pedestrians	28	Children	29	Bicycles
30	Snow/Ice	31	Wild animals	32	End all restrictions
33	Turn right	34	Turn left	35	Ahead only
36	Ahead or right	37	Ahead or left	38	Keep right
39	Keep left	40	Roundabout	41	End no overtaking
42	End truck overtaking	—	—	—	—

5. OpenCV Image Preprocessing Module

The Preprocessor class implements a configurable preprocessing chain: BGR -> Resize(64x64) -> Grayscale (optional) -> Gaussian Blur (optional) -> CLAHE (optional) -> Normalization (divide255 or minmax). All parameters are recorded in self.config for training/prediction consistency.

Output is uint8 grayscale (64,64) for HOG consumption; HSV feature mode retains BGR color input.

5.1 Preprocessing Pipeline

```
class Preprocessor:
    def __init__(self, img_size=64, to_gray=True,
                 gaussian_ksize=3,          # 0=disabled
                 clahe=True, clahe_clip=2.0, clahe_grid=8,
                 normalize="divide255"): # "divide255" | "minmax"
        ...

    def __call__(self, img_bgr: np.ndarray) -> np.ndarray:
        # 1) cv2.resize -> (64, 64)
        # 2) cv2.COLOR_BGR2GRAY (if to_gray=True)
        # 3) cv2.GaussianBlur (if gaussian_ksize > 0)
        # 4) cv2.createCLAHE -> apply (if clahe=True)
        # 5) divide255: float32/255 -> clip -> *255 -> uint8
        #    minmax:   cv2.normalize(NORM_MINMAX) -> uint8
        # Returns uint8 grayscale (64, 64)
```

5.2 Data Augmentation (Training Only)

augmentation.py provides random affine transforms, brightness/contrast adjustment, Gaussian noise, and blur, applied with 0.5 probability stacking 0-2 augmentations. Key constraint: NO horizontal flip (directional semantics would be destroyed – left turn becomes right turn, numbers/text reversed).

```
def random_affine(img, max_angle=10, max_shift=0.1, max_scale=0.1)
def random_brightness_contrast(img, brightness=0.2, contrast=0.2)
def gaussian_noise(img, sigma=5)
def gaussian_blur(img, ksize=3)
def apply_random(img, p=0.5): # randomly applies 0-2 augmentations
```

6. Feature Extraction: HOG, HSV Histogram and Feature Fusion

The system supports three feature modes managed by FeatureBuilder: HOG-only (1764 dims), HSV-only (64 dims), HOG+HSV fusion (1828 dims).

6.1 HOG – Histogram of Oriented Gradients

HOG describes shape and edge contours by counting local gradient orientation distributions. Traffic signs have stable geometric shapes (circle, triangle, octagon, rectangle) and internal patterns (numbers, arrows, symbols), making HOG ideal for this task.

Gradient computation: for each pixel, $G_x = dI/dx$, $G_y = dI/dy$, magnitude $m = \sqrt{G_x^2 + G_y^2}$, orientation $\theta = \arctan(G_y/G_x)$. The 64x64 image is divided into 8x8 pixel cells, each 2x2 cells form a block, blocks slide with 8-pixel stride. Each cell accumulates a weighted histogram over 9 orientation bins (0-180 degrees, unsigned gradient).

Parameter	Value	Description
winSize	64x64	Input image size, matches IMG_SIZE

Parameter	Value	Description
blockSize	16x16	Normalization block (2x2 cells)
blockStride	8x8	Block stride (1 cell)
cellSize	8x8	Cell size
nbins	9	Orientation bins (0–180 deg)
Feature Dim	1764	$(64/8-1)^2 \times (16^2/8^2) \times 9 = 7 \times 7 \times 4 \times 9$

6.2 HSV Color Histogram

Using only grayscale HOG loses color information (e.g., red prohibitory vs blue mandatory signs). The HSV histogram extracts the 2D distribution of hue H and saturation S, L1-normalized to a fixed-dimension color feature vector.

```
hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0,1], None, [8,8], [0,180,0,256])
hist = hist.reshape(-1).astype(np.float32) / (hist.sum() + 1e-6)
# -> 64-dim L1-normalized vector
```

6.3 Feature Fusion

FeatureBuilder selects the feature mode based on the 'mode' parameter:

- ☐ hog – HOG only (1764 dims), suitable for classes with distinct shapes
- ☐ hsv – HSV histogram only (64 dims), suitable for classes with distinct colors
- ☐ hog+hsv – Concatenation (1828 dims), fuses shape and color, typically yields best results

After fusion, StandardScaler is fit on training data and applied to all splits before classification.

7. Classification Models: SVM / KNN / Random Forest

Three classifiers are implemented for comparison: SVM (primary), KNN (baseline), Random Forest (comparison). All models are persisted via a unified save_bundle/load_bundle interface.

7.1 SVM – Support Vector Machine (Primary Model)

Uses RBF kernel SVC with probability=True for confidence output. RBF kernel handles the non-linear separability of high-dimensional HOG features. Default hyperparameters: C=10, gamma='scale'.

Multi-class (43 classes) via One-vs-One strategy.

```
from sklearn.svm import SVC
model = SVC(C=10, kernel="rbf", gamma="scale",
            probability=True, random_state=42)
model.fit(X_train_scaled, y_train)
```

7.2 KNN – K-Nearest Neighbors (Baseline)

KNN provides an intuitive baseline. Uses distance weighting: closer neighbors have higher weights. Default k=5. Prediction requires computing distances from the test sample to ALL training samples, making inference slower.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5, weights="distance", n_jobs=-1)
```


7.3 Random Forest (Comparison Model)

Random Forest is an ensemble of decision trees, insensitive to feature scaling. Feature importance can be inspected. Default `n_estimators=200`.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=200, max_depth=None,
                             random_state=42, n_jobs=-1)
```

7.4 Model Persistence

`model_manager.py` packages the classifier, `StandardScaler`, label mapping, feature config, and training summary into a single `.joblib` bundle. Predictor validates feature dimensions on load.

```
bundle = {
    "classifier": model,          # classifier object
    "scaler": scaler,           # fitted StandardScaler
    "label_map": {0: "Speed limit 20", ...},
    "feature_config": {"mode": "hog+hsv", "img_size": 64, ...},
    "summary": {"model": "svm", "feature_mode": "hog+hsv",
               "n_train": 62734, "feature_dim": 1828,
               "train_seconds": 123.4, ...}
}
joblib.dump(bundle, "svm_hog+hsv.joblib")
```

8. Training Pipeline and Hyperparameter Selection

Training strictly follows the no-test-set-leakage principle: `StandardScaler` is fit and classifier is trained only on the training set; validation set is used for hyperparameter selection; test set is used only once for final evaluation.

8.1 Training Flow

- 1) `load_train_data(root) -> (images, labels)`
- 2) `stratified_split -> train/val/test (80/20 or 70/15/15)`
- 3) `Preprocessor + FeatureBuilder -> X_train, X_val, X_test`
- 4) `StandardScaler.fit(X_train) -> transform all three`
- 5) `model.fit(X_train_scaled, y_train)`
- 6) `val_acc = evaluate(model, X_val_scaled, y_val)`
- 7) [optional] `GridSearchCV` on `X_train` only (`C`, `gamma`)
- 8) `test_acc = evaluate(model, X_test_scaled, y_test)`
- 9) `save_bundle(model, scaler, label_map, feature_config, summary)`

8.2 SVM Hyperparameter Selection

RBF SVM has two key hyperparameters: `C` (regularization strength) and `gamma` (RBF kernel width). Grid search on training set only (not touching val/test):

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
param_grid = {"C": [0.1, 1, 10, 100],
              "gamma": ["scale", "auto", 0.01, 0.1]}
grid = GridSearchCV(SVC(kernel="rbf", probability=True),
                    param_grid, cv=StratifiedKFold(3),
                    scoring="accuracy", n_jobs=-1)
grid.fit(X_train_scaled, y_train)
best_C = grid.best_params_["C"] # typically 10
best_gamma = grid.best_params_["gamma"] # typically "scale"
```

8.3 CLI Training Commands

```
# Train SVM (recommended HOG+HSV)
python -m traffic_sign_system.scripts.train --model svm --mode hog+hsv --C 10

# Grid search SVM hyperparams
python -m traffic_sign_system.scripts.train --model svm --mode hog+hsv --grid

# Train KNN / Random Forest
python -m traffic_sign_system.scripts.train --model knn --mode hog+hsv
python -m traffic_sign_system.scripts.train --model rf --mode hog+hsv
```

9. Model Evaluation and Results Analysis

The evaluation module outputs Accuracy, macro/weighted average Precision/Recall/F1, normalized confusion matrix, and error sample CSV. All metrics are computed on the test set (12,630 samples) to ensure objectivity.

9.1 Evaluation Metrics

Metric	Formula/Meaning	Usage
Accuracy	Correct / Total	Overall recognition capability
Precision (Macro)	Arithmetic mean of per-class Precision	Overall false-positive assessment
Recall (Macro)	Arithmetic mean of per-class Recall	Missed-detection assessment (unaffected by class imbalance)
F1 (Macro)	$2 * P * R / (P + R)$ macro average	Balanced score under class imbalance
Confusion Matrix	True class x Predicted class cross-tabulation	Locate specific confusion pairs (e.g., 30<->50 km/h)

9.2 Evaluation CLI

```
python -m traffic_sign_system.scripts.evaluate --bundle models/artifacts/svm_hog+hsv.joblib --data dataset/test/
# Output: metrics.json, confusion_matrix.png, errors.csv
```

Expected GTSRB 43-class SVM(HOG+HSV) test accuracy is approximately 94%-97% (depending on preprocessing parameters and data split). HOG-only is ~92%-95%, HSV-only ~55%-70%, fusion typically improves by 1-3 percentage points.

10. SVM vs KNN vs RF Multi-Model Comparison

Comparison of three classifiers on identical features (HOG+HSV, 1828 dims) and data split (random_state=42). Metrics include accuracy, F1, training time, prediction time, and model size.

Model	Val Acc	Test Acc	Train Time	Predict Time	Size	Characteristics
SVM (RBF)	~96%	~95%	2-5 min	seconds	~10-50 MB	Best on high-dim HOG features

Model	Val Acc	Test Acc	Train Time	Predict Time	Size	Characteristics
KNN (k=5)	~93%	~92%	Instant (lazy)	Slow (search)	~200 MB+	Must store all training features for prediction
RandomForest	~91%	~90%	1-3 min	milliseconds	~50-200 MB	Provides feature importance for analysis

Comparison CLI:

```
python -m traffic_sign_system.scripts.compare --features models/artifacts/features_hog+hsv.npz
# Output: comparison.csv, comparison.png, confusion_matrices.png
```

11. Error Samples and Confusable Class Analysis

error_analysis.py counts (true, pred) pair frequencies, outputs Top-10 confusion pairs and per-class Recall. Direction matters: A->B and B->A are distinct error patterns.

11.1 Common Confusion Patterns

Confusion Pair	Likely Cause
30 km/h <-> 50 km/h	Similar round shape, only digit differs; HOG digit detail resolution limited
Yield -> General caution	Both triangular red-border signs, similar contour
No overtaking -> End no overtaking	Identical shape, only internal diagonal line differs; gray HOG misses color
Left turn -> Ahead only	Arrow direction differences less significant at 64x64 resolution
80 km/h -> 60 km/h	Digit glyph similarity (8 vs 6), especially at low resolution

11.2 Improvement Directions

Based on error analysis, the following improvements can be made:

- ☐ Fuse HSV color features: red prohibitory vs blue mandatory vs white speed limit can be distinguished by color histogram
- ☐ Increase input size: 64->96 or 128 preserves more digit/symbol detail
- ☐ Class balancing: oversample minority classes (SMOTE) or adjust class weights (class_weight='balanced')
- ☐ Feature enhancement: try LBP (Local Binary Patterns) or SIFT bag-of-words as supplementary features

Error analysis CLI:

```
python -m traffic_sign_system.scripts.error_stats --bundle models/artifacts/svm_hog+hsv.joblib --data dataset/test/
# Output: top_confusions.csv/png, errors_per_class.csv/png
```

12. PyQt5 Graphical User Interface Design

MainWindow is based on QMainWindow with a left-canvas + right-control-panel layout. Supports three tabs (Image/Video/Camera), avoids UI freezing via QThread background threads, and drives video display via QTimer(33ms).

12.1 Main Window Layout

Menu: File(F) Model(M) Video/Camera(V) Help(H)			
+ Image	Video	Camera	[Control Panel]
+-----+ +-----+ +-----+			Model Path:
Original	Video	Camera	[] [Browse]
Image	Stream	Feed	[Load Model]
+-----+			
+-----+			[Image Mode]
Preproc.			[Select Image]
(Gray)			[Recognize]
+-----+ +-----+			[Scene Detect]
+-----+			[Clear Image]
Result	[Progress]		
Overlay			[Video/Camera]
+-----+			[Pause/Stop]
+-----+			[Save Result]
			[Result]
			ID: --
			Name: --
			Confidence: --
+-----+			
Status: Ready Time: -- ms Model: SVM Feature:...			
+-----+			

12.2 Feature Table

Feature	Operation	Implementation
Image Recognition	Select image -> Recognize	PredictWorker(QThread) background inference
Video Recognition	Select video -> Auto play	QTimer(33ms) per-frame VideoRecognizer
Camera Recognition	Select camera index -> Open	CameraRecognizer center-ROI fixed-box classification
Scene Detection	Click Scene Detect	DetectWorker(QThread) HSV+contour+classification
Pause/Resume	Click Pause button	Stop/Restart QTimer
Save Result	Click Save Result	cv2.VideoWriter writes .mp4

13. Traditional Vision Detection Extension (SignDetector)

SignDetector is a demonstration extension for locating candidate traffic sign regions in full road scenes. It uses a classic pipeline of HSV color segmentation + morphological processing + contour geometric filtering, then passes filtered candidate ROIs to the trained classifier.

!! This method has limited stability under complex lighting, occlusion, motion blur, or partial sign visibility. It is intended to demonstrate traditional computer vision processing pipelines and is NOT suitable for safety-critical applications.

13.1 Detection Pipeline

1) HSV Color Masking:

Red (two-segment union): $(0, 80, 80) - (10, 255, 255) \cup (170, 80, 80) - (180, 255, 255)$
 Blue: $(90, 80, 80) - (140, 255, 255)$

2) Morphological Processing: Elliptical kernel (5x5) opening -> closing

3) Contour Filtering (all thresholds configurable):

<code>min_area = 400 px²</code>	Minimum pixel area
<code>max_area_ratio = 0.3</code>	Upper bound as fraction of total image
<code>min_aspect = 0.5</code>	Minimum width/height ratio
<code>max_aspect = 2.0</code>	Maximum width/height ratio
<code>min_circularity = 0.4</code>	Minimum circularity = $4\pi \cdot \text{area} / (\text{perimeter}^2 + 1e-6)$

4) Crop ROI -> Predictor.predict()

5) Filter results with confidence < 0.5

13.2 Design Rationale

The detection strategy follows 'prefer missed detections over false positives': high circularity threshold filters non-circular noise, area and aspect ratio exclude elongated/oversized regions, low-confidence results are discarded. Some signs may be missed, but false detection rate is kept low.

In the GUI, click the "Scene Detection" button to run SignDetector on the current image or video frame. Results are annotated on the display with colored bounding boxes (red/blue) + class name + confidence.

14. Improvement Directions and Outlook

14.1 Feature Level

- ☐ Introduce LBP (Local Binary Patterns) texture features to better describe internal sign symbols
- ☐ Try multi-scale HOG or pyramid HOG to improve adaptability to different sign sizes
- ☐ Extend color features from H/S 2D histogram to include V (brightness) in a 3D histogram
- ☐ Use PCA or LDA to reduce high-dimensional HOG features (1764 dims), reducing redundancy and speeding up training

14.2 Model Level

- ☐ Try LinearSVC for faster training with large sample counts (GTSRB 39k samples feasible)
- ☐ Introduce XGBoost/LightGBM gradient boosting trees as comparison models
- ☐ Use Stacking ensemble to combine SVM + KNN + RF predictions

- ☐ Explore CNN approaches (e.g., lightweight MobileNet/ResNet18) for quantitative comparison with traditional methods

14.3 System Level

- ☐ Replace SignDetector HSV localization with CNN-based object detector (e.g., YOLO/SSD)
- ☐ Add batch testing mode: auto-run on entire test set and aggregate results
- ☐ Support model hot-swapping: switch between different bundles without restarting GUI
- ☐ Output PDF report: one-click generation of comprehensive experiment report with all evaluation charts

15. Limitations and Conclusion

15.1 Current Limitations

Limitation	Description
Fixed feature dimension	HOG window 64x64 requires input to strictly be this size; no multi-scale support
Simple color feature	H/S 2D 8x8 histogram may be insufficient to distinguish similarly-colored signs
Low detector robustness	Pure HSV color segmentation adapts poorly to varying lighting, shadows, and occlusions
No GPU acceleration	SVM training runs on CPU; CNN deep models not yet integrated
Class imbalance	GTSRB class sample counts vary widely (hundreds to 2000+), potentially affecting minority class recall
No real-time video detection	SignDetector HSV+contour processing is too slow for real-time video detection

15.2 Conclusion

This project built a complete traffic sign classification and recognition system based on OpenCV + HOG + SVM, covering all stages from dataset management, image preprocessing, feature engineering, model training and comparison, model evaluation and error analysis, to PyQt5 GUI interaction and traditional vision detection extension. The system uses modular design with decoupled, configuration-driven, and highly reproducible components.

Three technical pipelines – HOG-only, HSV color histogram-only, and HOG+HSV fusion – provide a clear comparison framework for feature engineering experiments. The unified training and comparison mechanism for SVM/KNN/RF classifiers provides evidence-based model selection. The PyQt5 GUI supports image, video, and camera recognition modes, plus the traditional HSV detection extension, forming a complete demonstration system.

This system is suitable as a complete computer vision course design solution, covering core knowledge areas including image processing, feature extraction, machine learning classification, model

evaluation, and desktop GUI development, with both theoretical depth and practical engineering value.

--- END ---